

This course has already ended.

« Chapter 14.0: Presentation of the Spring Course (... Chapter 14.2: UML Modeling » (https://plus.cs.aalto.fi/studio_2/k2022/w14/)
CS-C2120 (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 14 (https://plus.cs.aalto.fi/studio_2/k2022/w14/)
/ Chapter 14.1: Planning

Chapter 14.1: Planning

About this page:

Topics: How to get started in designing a program.

What Will I Do? Read (Basic) Methods for Designing Programs.

Point Value: 0.

Indicative difficulty level: Moderate.

Rough Estimate of Workload:? Hour.

Related Projects: None.

Chapter 14.1: Designing a Program

Up to now, programming and design work on the course have been carried out on the basis of the ready-made documentation, and the implementation work has largely been done for existing projects (except for the text game assignment). As stated in the autumn course, starting programming from a completely clean table is far less common than developing an existing program.

As you carry out your first big project, you will notice that the editing an existing program is not straightforward either. Bigger changes to an existing program are themselves projects that are equally demanding to design. Editing a working program may actually be more demanding than implementing a completely new program, because one has to understand the whole program logic and structure very well.

In Chapter 16.2 we will look at unit testing, which can reduce the risk that something gets unnoticed, when editing the program.

What is design?

Program design is, on one hand, a process in which the requirements and wishes concerning the program are considered to generate the first plan on which the implementation starts. On the other hand, designing is also an activity that continues and is needed through the entire project implementation and life cycle. Although designing is therefore much needed at the very beginning of the project, design and implementation alternate during the project life span.

In this section, we will look at the initial design of the program through an example project.

Note. In this course, we consider mainly the object-oriented design point of view, i.e., conceptual modelling of the problem domain and class structure design. Program design covers also many other aspects, such as, software architecture, usability and user interface design, software performance, choice of data structures and algorithms, etc. They are covered in other courses.

Example Project: Timeliner

A verbal description of the program

The goal of Timeliner project is to solve the everlasting problem of students and teachers – overlapping times of high workload.

Work commitments overlapping on the same days or even the same hours cause difficulties for both students, teachers and planning officers. We will plan and implement a tool that allows teachers and others to place lectures, examinations, exercises, appointments, projects, usually all study-related activities in student's calendar, in order to display them for both students and other teachers. When the tool also “knows” which courses students take in the same semester, it can show students' other course deadlines for the teacher when one's own course is still in planning phase. With these features, the program is a kind of community calendar. However, it becomes an interesting tool when students are given the opportunity to split their project work hours into the parts they want and place them within an interval between the project publication time and the deadline by dragging them to the desired places. Of course, some of the activities like Wappu cannot be transferred or performed in parts.

The program keeps track of the workload and any fixed activities planned for working days and working weeks. In addition, the program will present this to both students themselves and teachers and planning officers.

Although it is most sensible to implement the actual project as a web service, we will build a first prototype to test the functionality of the concept as a desktop application. The web-based project can be implemented on the course CS-C3170 Web Software Development  (<https://mycourses.aalto.fi/course/search.php?search=CS-C3170>).

Requirement Definitions and Modeling

In real life, a much more precise definition of the requirement would be made before the start of work, but we are now studying modeling and implementation of small to medium size programs. You can learn more about requirements specification and modeling, for example on courses CS-C3180 Software Design and Modelling [🔗](https://mycourses.aalto.fi/course/search.php?search=CS-C3180) (https://mycourses.aalto.fi/course/search.php?search=CS-C3180) and CS-C3150 Software Engineering [🔗](https://mycourses.aalto.fi/course/search.php?search=CS-C3150) (https://mycourses.aalto.fi/course/search.php?search=CS-C3150)

How should we proceed?

For a slightly larger program it is a good idea to design and model the program structure and to consider the different options before proceeding to coding at the keyboard. This will save a lot of unnecessary work.

There is no overall best way to design everything, just like there is no best way to implement a program. Designing also continues through the implementation phase of the program, because customers can update their requirements. Or, when implementing the program, we may find out that it is worthwhile to change the design in order to achieve a better outcome or even to be able to complete the project at all.

Ad hoc modeling - Verb-Noun method

The verb-noun method (Noun method, Noun-phrase approach, Noun-verb method etc.) is often presented as a technique to make the first coarse model of the program class structure. The model is not meant to be a complete solution to the problem, but to provide a starting point from which the design is to be elaborated.

There are a lot of different variants of the method, but its first steps are basically the following:

1. A short but accurate verbal description of the program's functional requirements is written. Things that are related to the user interface are deliberately left out at this stage.
2. We search for the description of all the nouns and try to group these to form initial classes and their instance variables (fields).

- It's easy to see that some of the nouns can be rejected directly. In some of the classes, this matter will only be discovered later.
- Things for which you can create own useful instance variables are potential classes. Other things are often class instance variables that refer to the instances of other defined classes or are described using the existing types offered by Scala.
- Later on, it may turn out to be necessary or useful to create "technical auxiliary classes" that have no direct correspondence in the "real world". However, in the beginning of building the class structure, it is important to begin with the application domain concepts.

3. We are looking for verbs in the text. Among these, we can identify potential methods

- It's a good idea to get rid of the unnecessary verbs right away.
- Then place the functions (methods) in the context where their data is processed.
- One can get tips on how to place the methods by looking at what information they are dealing with. It is worth considering which variables the functions should affect and what variable values they need for their working.

What about the other words?

Some of the variations of this method also consider adjectives or adverbs, which are sometimes modelled with instance variables, sometimes with traits.

The final result is a preliminary model, which still lacks much information. Once the first design has been achieved, it is being modified and checked to see if it can really meet the original requirements.

4. The initial classes certainly will lack the instance variables and the necessary methods that can be added if needed. Also complete classes are certainly missing. Existing classes may need to be combined or split into pieces, etc. Such changes can be carried out at this stage with quite a small effort.

Are there any useful tools here?

No dedicated tools are really needed for this step. Perhaps the most powerful tool for searching and sorting words is to use a whiteboard and post-it stickers. Words can be typed into stickers and grouped by moving them on the board. Equally, the same can be done on paper or in a spreadsheet program, where each class can make its own column and we can drag words from one place to another.

When the end result looks good, a UML (Unified Modeling Language) class diagram can be drawn from the template. This is a more convenient way to describe relationships between classes and to get an overall view of the program. In the autumn course, such charts have been used many

times, even though this was not specifically mentioned. We will learn about UML charts in the next section.

Example: Timeliner modeling with a noun method

We mark nouns in red, but do not mark the same nouns multiple times. If necessary, we specify some nouns by adding the attributes.

The **goal** of **Timeliner** project is to solve the everlasting **problem** of **students** and **teachers** – overlapping **times of high workload**.

Work commitments overlapping on the same **days** or even same **hours** cause **difficulties** for both students, teachers and **planning officers**. We plan and implement a **tool** that allows teachers and **others** to place **lectures**, **examinations**, **exercises**, **appointments**, **projects**, usually **all study-related activities** in student's **calendar**, in order to display them for both students and other teachers. When the tool also “knows” which **courses** students take in the same **semester**, it can show students' other course **deadlines** for the teacher when one's own course is still in **planning phase**. With these **features**, the **program** is a kind of community calendar. However, it becomes an interesting tool when students are given the **opportunity** to split their **project work** hours into the **parts** they want and place them within an **interval** between the project **publication time** and the deadline by dragging them to the desired **places**. Of course, some of the **activities** like **Wappu** cannot be transferred or performed in parts.

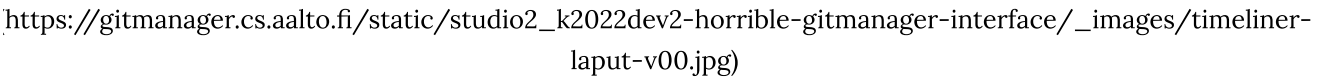
The program keeps track of the **workload** and any **fixed activities** planned for **working days** and **working weeks**. In addition, the program will present this to both students themselves and teachers and planning officers.

https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/_images/nouns.png

Nouns (singular form):

goal, problem, student, teacher, times of high workload, work commitment, day, hour, difficulty, planning officer, tool, other, lecture, examination, exercise, appointment, project, all study-related activities, calendar, course, semester, deadline, planning phase, feature, program, opportunity, project work, part, interval, publication time, place, activity, Wappu, workload, fixed activity, working day, working week.

The nouns are written on the stickers and placed for grouping on the board (do not worry that the words here are in Finnish).



All identified nouns will probably not be equally useful when identifying program classes. In addition, it is possible that some important classes are not even directly mentioned. How do we find the right classes with the help of the stickers?

Some of the nouns describe time. Let us add all time related stickers to the same group. Let us insert an attribute to noun *interval* that specifies the time between the publication and the deadline dates.

day, hour, semester, deadline, planning phase, interval (between publication time and deadline), publication time, working day, working week.

student, teacher, planning officer, other (person)

Calendar:

The third group is related to calendars. Let us specify that *place* is a place in the calendar, as it is not an arbitrary place.

calendar, place (in calendars)

Doing/working:

The fourth and largest group is related to activities people do, which take time and are supposed to fit into calendars. We also denote *part* to be focusing on project work to distinguish it from other parts.

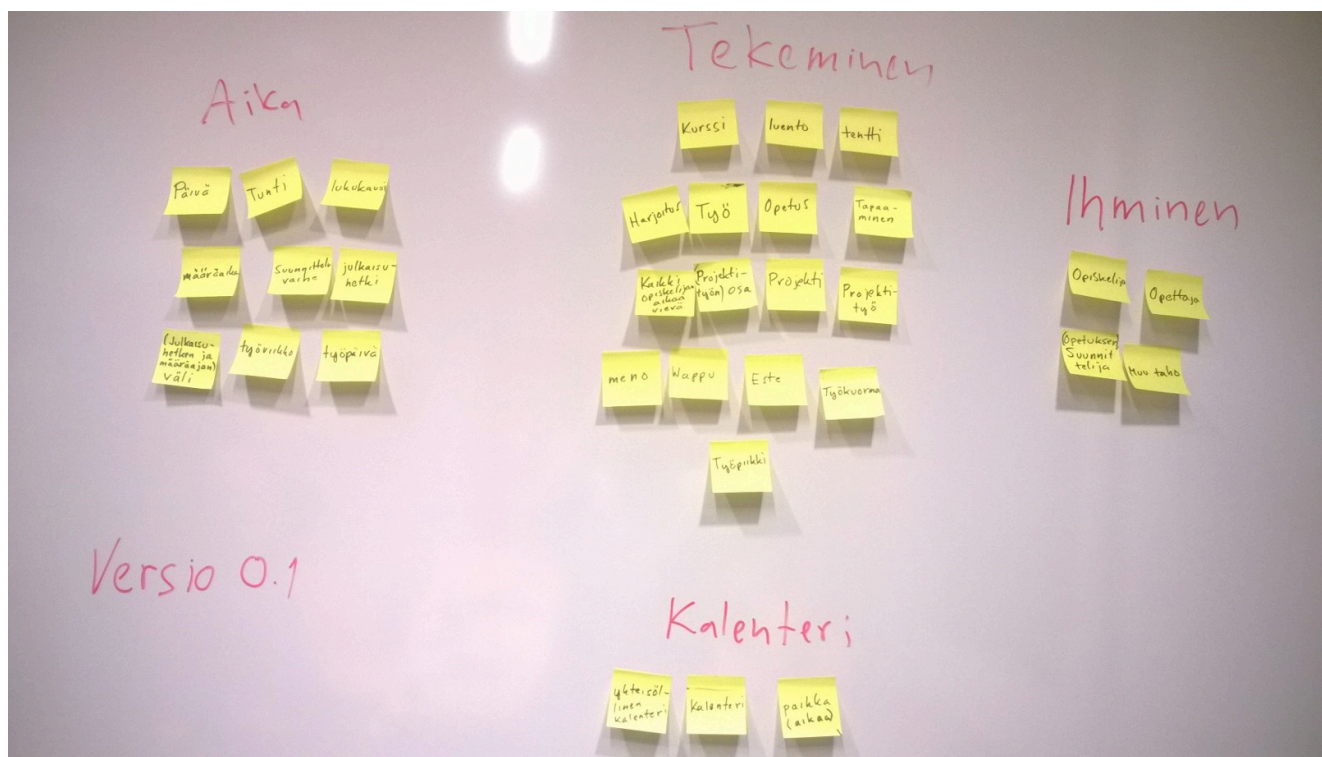
times of high workload, course, lecture, examination, exercise, appointment, project, all study-related activities, part (of project work), project work, workload, activity, fixed activity, Wappu

Abandoned:

A set of nouns was left over. They do not seem to be relevant because they are not related to the problem area being dealt with, but to this design process, or they are too general terms. Discard these words at this point.

goal, problem, work commitment, difficulty, tool, feature, program, opportunity

At this point, we have the following grouping: (Rough translation: Aika = Time, Tekeminen = Working, Ihminen = People, Kalenteri = Calendar, Versio = Version).



https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/_images/timeliner-laput-v01.jpg

We look at the groups more closely and try to identify the classes.

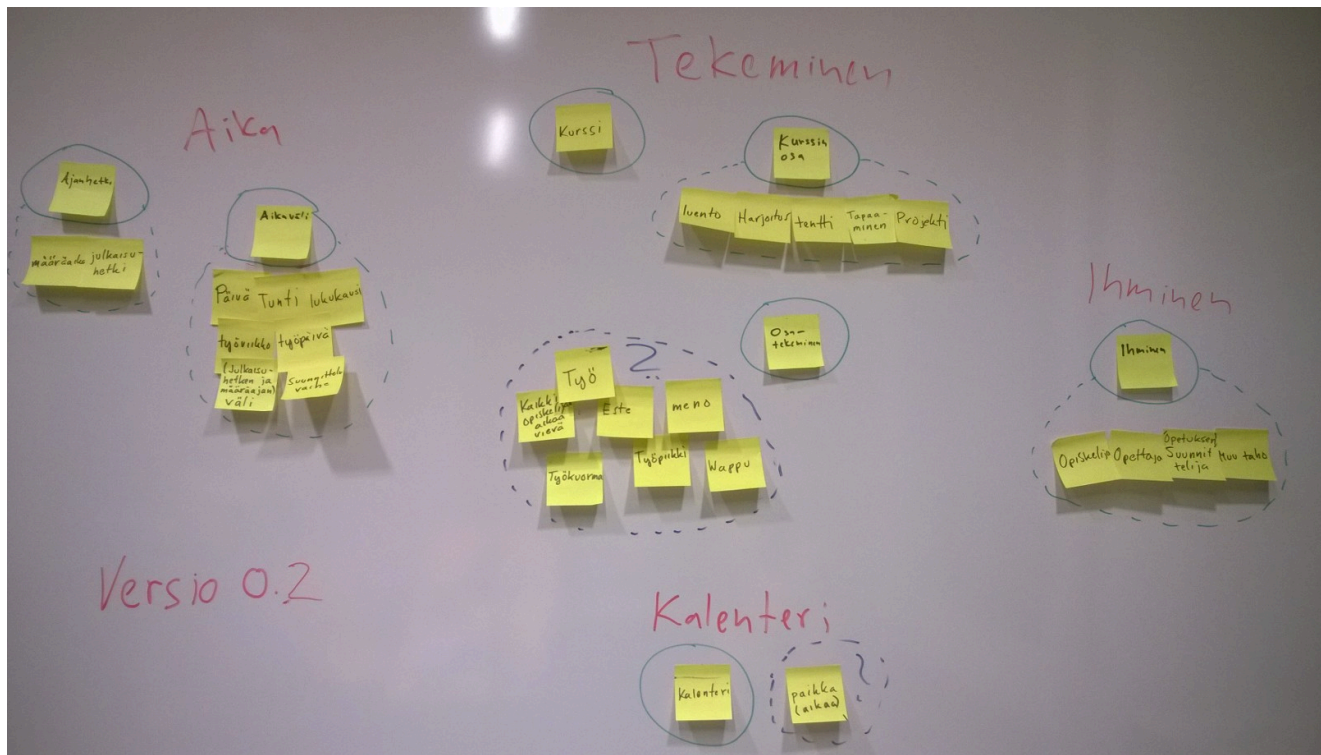
Time-related nouns appear to describe either individual time periods (*deadline*, *publication time*) or time slots (*day*, *hour*, *semester*, *working day*, *working week*, *planning phase*, *time between publication time and deadline*). Each time period has a starting time and ending time. It seems that we need at least the classes of *time* and *time interval*. With good luck, they may even be available as a library class. It is not clear whether we need classes, for example, for *days*, *hours*, and *semesters*, or are these just a mere *interval*. If, for instance, it turns out that *day* has some fields *time interval* does not have, it is justified to set the *day* as a subclass of the *time interval*. Similarly, the *deadline* and *publication time* could be subclasses of *time*, but it is not yet clear whether they need to be defined. Let us not make a final decision here yet. In general, it is not advisable to make definitive choices in unclear situations.

What about the nouns related to working? These are mainly related to studying and teaching, although there are also other things, such as *project work*, *fixed activity*, *workload*, and *Wappu*. Let us first try to get a sense of the classes related to teaching and studying.

Based on the description of the problem, an important noun seems to be a *course*. It seems to be related to a *teacher* and a group of *students* who have taken the course. Based on the description of the problem, a *lecture*, *exam*, *exercise*, *meeting*, and *project work* are part of a *course*. One *course* may certainly have a variable number of these. We recognize a new class *course part* that is not mentioned in the description of the problem, but which describes any of the course activities involved, such as a *lecture* or an *exam*. We may be able to determine that a *lecture* etc. are subclasses of *course part*. It is not yet clear whether this is profitable, so let us not make a final decision yet. The description of the problem shows that, for example, *project work* can still be divided into *parts*. Suppose other *course parts* can be divided into parts. This allows you to drop out *project work* from the group and replace it with a more general class *component*.

We can still make the *calendar* and the *people* their own classes. A *teacher*, a *student*, a *planning officer*, and *others* may, if necessary, be subclasses of *people*.

As a result of this step, we have obtained the breakdown as shown below. In the picture, we have surrounded the classes identified by a uniform green line. Within the green dash line there are their possible subclasses (we are not yet sure if we need them). The blue dash line surrounds by nouns that we have not used yet and have not abandoned (again the words are in Finnish but you get the idea. Group titles: Aika = Time, Tekeminen = Working, Ihminen = People, Kalenteri = Calendar, Versio = Version).



https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/_images/timeliner-laput-v02.jpg)

classes

At this point, we have a preliminary list of classes:

people, time, time interval, calendar, course, course part, component

Verbs

Let's look at verbs and mark them in green.

The goal of Timeliner project is to **solve** the everlasting problem of students and teachers – overlapping times of high workload.

Work commitments **overlapping** on the same days or even same hours **cause** difficulties for both students, teachers and planning officers. We **plan** and **implement** a tool that **allows** teachers and others to **place** lectures, examinations, exercises, appointments, projects, usually all study-related activities in student's calendar, in order to **display** them for both students and other teachers. When the tool also "**knows**" which courses students **take** in the same semester, it can **show** students' other course deadlines for the teacher when one's own course **is** still in planning phase. With these features, the program is a kind of community calendar. However, it **becomes** an interesting tool when students are **given** the opportunity to **split** their project work hours into the parts they **want** and **place** them within an interval between the project publication time and the deadline by **dragging** them to the desired places. Of course, some of the activities like Wappu cannot be **transferred** or **performed** in parts.

The program **keeps track** of the workload and any fixed activities planned for working days and working weeks. In addition, the program will **present** this to both students themselves and teachers and planning officers.

https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/_images/verbs.png

The verbs are written in the basic form. verbs:

solve, overlap, cause, plan, implement, allow, place, display, know, take, show, be, become, give, split, want, drag, transfer, perform, keep track, present

Likewise, we can reject some of the verbs already at this stage. Verbs *solve* and *become* describe the program's development-process-related activities and not actions taken by the program. *Cause* is used in a context "cause difficulties" which we do not want to implement, so we discard it. *Allow* is not an independent verb here, so we discard it, too. The *be* verb describes a wide range of things, but they are not an activity, so in some cases it can be ignored. In other words, the meaning of the verb is (without context) multifaceted; so it is a good idea to write attributes to clarify what the verb means in each case.

Now, the remaining verbs should form methods.

The verb to *place* is used to describe the problem, as follows: "...teachers and others to place lectures, examinations, exercises, appointments, projects, usually all study-related activities in student's calendar." At this point, we find out that it might be better to use the "study-related activities" noun to describe something more general than the part of our course that we have previously defined. There is a noun *work commitment* which we discarded but which fits here. Therefore we define the class *work* and make it a superclass of the class *component*. We can match verb *place* with method *placeWork*, which takes as parameters a *calendar* and *work*. Of course, we also want to be able to remove *work* from the calendar, so accordingly we determine the method *removeWork*. We note at the same time that the *calendar* is a container class for instances of class *work*.

The verb *display* is used as follows: "...display them [study activities] for both students and other teachers". From the program perspective, this view means that the *work* in the calendar can be read by some method. Let us name this method *readWork*, and it takes the *calendar* as a

parameter and returns the *works* in the calendar.

The verb *take* is used in the description like this: "...which courses students take in the same semester". What does it mean for our program that a student takes the course? Certainly the works included in the course (lectures, exams, exercises, etc.) is placed in the student's calendar.

At this point, we recognize that a student needs a calendar where one can add work. We define the method *takeCourse*, which receives as a parameter a *student* and a *course* and adds the course works to the student's *calendar*. If each student has a calendar including his/her works, should all people have a corresponding personal calendar? We can associate a personal calendar for each person. But which calendar is the one the teacher uses to place the course work? We might perhaps place these in the teacher's personal calendar, but it may be better to define for each course a unique calendar including the course work.

The verb *show* seems to be used, as follows: "...show students' other course deadlines for the teacher when one's own course is still in planning phase". It is therefore a matter of displaying deadlines. Let us determine the method *showOtherDeadlines*, which has as a parameter the *course* and which returns as a result a set of *time* slots. These deadlines are therefore time-limits for different tasks. It can therefore be concluded that class *work* is associated with *deadlines*. We have finally solved the previous open problem of whether *deadline* should be a class or not. It is not necessary; it can be a feature of an instance of the class *work*. Likewise, we can make a feature from *publication time* mentioned in the text. The deadline and publication time are therefore work-related features that define the time limit for the job to be performed. Since *publication time* and *deadline* appear in this pair, we can replace them by one feature *time limitation* that is an instance of the class *time* slot. *Work* is also usually associated with estimated duration, which may be either shorter or as long as the *time limitation*.

The verb *split* is used in the description, as follows: "students are given the opportunity to split their project work hours into the parts they want and place them within an interval between the project publication time and the deadline". This does not necessarily have to be part of a *project work* but it could be part of any work. We determine the method *splitWork* that takes the *work* as a parameter and produces a set of class *component* instances. But how does the method know how the split should be carried out? At this stage, we do not specify any algorithm for this, but we prepare for different ways by defining the class *division policy*, whose instance is also given as a parameter for the method *splitWork*.

The verb *transfer* is used as follows: "Some of the activities like Wappu cannot be transferred or performed in parts." The activity is probably the synonym for *work*. Transferring can be done by using previously defined methods *removeWork* and *placeWork*. However, the text reveals that this is not always permissible. We add into the class *work* fields which denote whether the work can be *split* or *transferred*.

Our plan is not at all complete yet. However, the description above gives a picture how the planning process can proceed from the analysis of concepts and related actions in the task towards identifying classes, instance variables and methods, which can be used to describe them in the program. We continue now the analysis from another point of view.

User stories

In Agile Methods, the requirements of the program can be described as so-called “user stories”. The user stories are more informal than use cases  (https://en.wikipedia.org/wiki/Use_case), which are another common way of describing the usage scenarios. Even though a user story does not need to be a single sentence, it’s often deliberately short, and it’s probably a story that could be easily used to form a test case for the program. The user story is usually written by the customer, while the use case description is written in collaboration between the project developer and the customers.

Examples of *user stories* from our *Timeliner* project:

1. "As a teacher I would like to create an exam for my course."
2. "As a teacher, I want to edit the workloads of the assignments of my own courses. Others should not be able to edit my courses."
3. "As a student, I want that when I choose a course, all its events will appear in my calendar."
4. "As a student, I want to be able to share the project work hours for several weeks."
5. "As a student, I'd like the changes that I made to be saved automatically without that I need to save them explicitly."
6. "As a student, I would like the program to notify me when my assignment deadline is approaching."
7. "As a planning officer, I would like to see the workload of the whole annual cohort in a degree program in some way."
8. "As a teacher, I would like to have a warning if someone places a compulsory activity close to some task in my course."
9. "As a teacher, I would like to see when students plan to take my course."

Agile methods do not focus on working with program design before implementation, although some kind of first model of the program must be created. Typically, the philosophy of these methods is that the program design both can and should evolve during the development of the program.

Next, we will learn about the Responsibility-Driven Design method and CRC cards and try out with a few of the above listed user stories, how well the classes created by the noun method work in this context.

CRC Cards and Responsibility-Driven Design

CRC, the Class-Responsibility-Collaborators method, (Beck, Cunningham 1989)  (<http://c2.com/doc/oopsla89/paper.html>) is based on named CRC cards, respectively. Each card has three sections, a class name at the top of the card, class responsibilities on the left side of the card, and class partners on the right. Responsibilities are the things that the class

described in the card promises to take care of for other classes. And when a class needs information or functionality that another class provides, these are recorded in the right-hand field.

RDD, Responsibility-Driven Design  (http://en.wikipedia.org/wiki/Responsibility-driven_design) is an object-oriented design method that emphasizes behavior and works well with CRC cards. The process of creating cards and the functionality of the classes they describe is a kind of card game. When the initial classes exist, we can experiment with their activities by performing different use cases. In Section 2.1 (<https://plus.cs.aalto.fi/o1/2020/w02/ch01/>), we watched how the classes Window, Category, and Experience discussed with each other so that the new experience was added to the program's memory and the desired category. CRC cards are used in experimentation very similarly.

Let's try

Let us choose the first one from the Timeliner project user stories described above. As the story proceeds, the cards will be filled at the same time: responsibilities and class relationships will be added, as well as completely new classes. Similarly, we can make changes to existing classes, delegate responsibilities, split classes into smaller classes or merge them, etc.

<i>"As a teacher I would like to create an exam for my course."</i>

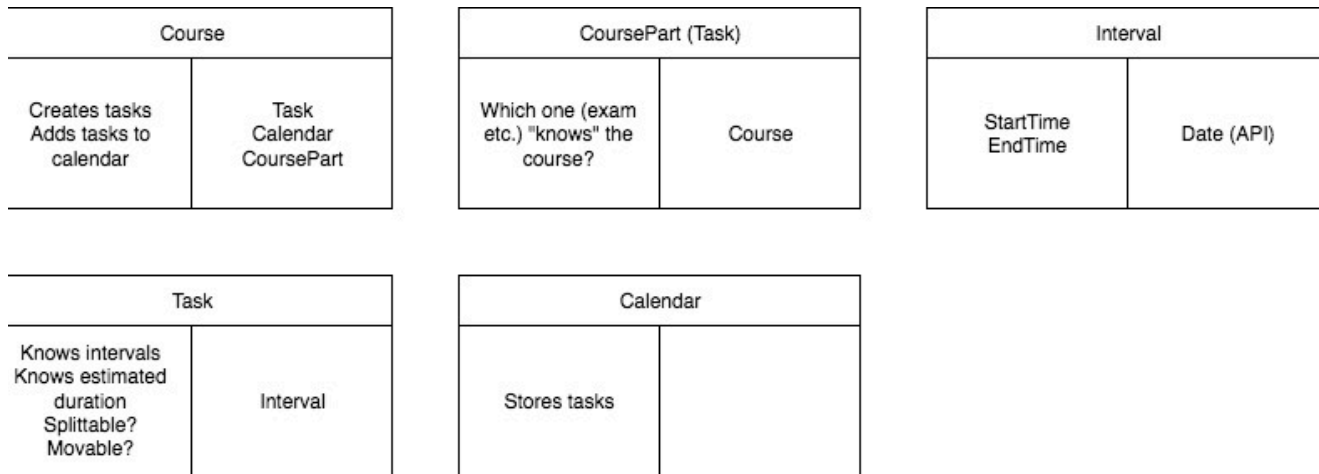
We ignore everything related to the user interface at this moment and we assume that initially we have a teacher's course object.

The Exam is part of the course (CoursePart). Thus, we need an entity that is able to create parts of the course. The most natural thing would be that this is taken care of by the Course object. (We mark this in the Course class responsibilities.)

Emm... The course object has its own Calendar, and the Exam should probably be added to this calendar. (We mark Calendar as the Course class associates.) As stated earlier, Work is the super class of CoursePart and the work includes a time limit in which the work is done and the estimated duration. (We record this in the cards.) It is further noted that, in the case of the exam, the estimated duration is the same as the length of the time limit.

Is there anything special about the exam? In the context of the substantive method, we noted that the student will place and share the course work on his/her own calendar. When it concerns participating in the exam, this must not be possible, so CoursePart must record what is divisible and transferrable. (We write these things for the responsibilities of CoursePart) es The story now needs to be re-examined so that it one "plays" the tasks of each class, and all essential things have been considered and recorded. More experienced users of the method recommend to avoid hurry.

After the first story, the cards look like this:



(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/_images/CRC_example.jpg)

It is clear that there is still a lot to be done in the classes. The exercise can be continued independently or with a pair. The outputs, i.e. modelled classes, will be used in the task of the next section where we draw a UML diagram of them.

Summary

- Program design is a process that proceeds through the whole program life cycle.
- Designing a program is not easy or straightforward, and it is learned primarily by doing it. Experienced programmers have (in their memory) ready-made solutions, schemas, which they can recognize in the problem area when designing a program, and they can then apply the known structures and methods.
- The Verb-Noun method provides one way to find a starting point for design. It generates an initial draft for the required classes.
- Responsibility-driven design is a method in which user-described activities can be tested in a simulation and use the gained information to refine the design of the required classes.

About future

- In the next section we introduce the UML class diagram that can visualize and document the designed class structure.
- In Section 14.3 we consider how we begin to implement the designed program.

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Time spent: (*) Pakollinen Please estimate the total number of minutes you spent on this chapter (reading, assignments, etc.). You don't have to be exact, but if you can produce an estimate to within 15 minutes or half an hour, that would be great.

20

Show anyway

"I feel that I have understood the most important things in this chapter." (*) Pakollinen

- ☒ fully agree
- ☐ somewhat agree
- ☐ somewhat disagree
- ☐ fully disagree
- ☐ I'm unable to answer or don't want to comment.

Submit an update

